

False-Positive-Constrained IoT Intrusion Detection

A reproducible pilot study using a fixed CICIoT2023 shard

Wisam Makki Salim
Computer Engineering | Network Security and Resilient Systems
Al-Iraqia University, Baghdad, Iraq | ORCID 0009-0000-6998-3912

Study aim

This pilot examines a practical question in binary IoT intrusion detection: whether an operating point selected to limit false alarms on validation data retains that behavior on unseen data. The emphasis is on data quality, leakage control, and the cost of changing the decision threshold rather than on proposing a new classifier. The random split is treated as a pilot evaluation, not as a simulation of future deployment traffic.

Research question

Can validation-based threshold selection retain high attack recall under a 1% false-positive-rate constraint, and does the chosen operating point continue to meet that constraint on a held-out partition?

Main finding

The selection rule chose a Random Forest threshold of 0.59853. On the held-out partition, attack recall was 98.167% and the false-positive rate was 1.208%. The conditional 95% interval for the false-positive rate was 0.846% to 1.601%. The point constraint observed during validation therefore did not remain satisfied on the held-out data.

Implication

A threshold that appears acceptable during model selection can cross the same false-alarm limit when it is evaluated once on unseen data. This pilot therefore motivates a decision rule that accounts for uncertainty and distribution change rather than relying only on a validation point estimate.

Scope

The evidence is limited to one randomly partitioned benchmark shard. It does not establish temporal robustness, production readiness, or cross-network generalization.

Data and methods

CICIoT2023 was obtained from the Canadian Institute for Cybersecurity. The dataset documentation describes 105 IoT devices and 33 attacks in seven categories. This pilot uses **Merged01.csv** as a fixed shard (712,311 rows and 39 predictors), identified by SHA-256 8b43d6552a8cafd3b0ca2cedf6464ca3fe644d7fc9bb5dfe906368f1542792fe. Raw data are not redistributed.

Data audit

Audit item	Observed	Consequence for this study
Exact duplicate rows	167,368 (23.50%)	Avoid splitting identical rows across partitions
Duplicate feature rows	194,204 (27.26%)	Use feature vectors as the sampling unit
Conflicting binary-label groups	16	Exclude these groups from the primary analysis
Attack prevalence	97.673%	Do not rely on accuracy or attack PR-AUC alone
Missing / infinite cells	22 / 14	Convert infinities and fit imputation on development data

Analysis design

After binary label conversion, feature groups with conflicting labels were removed and one representative of each remaining feature vector was retained. The resulting 518,091 observations were split 60/20/20 into development, validation, and held-out partitions using seed 20260920. Imputation and scaling were fitted on development data only.

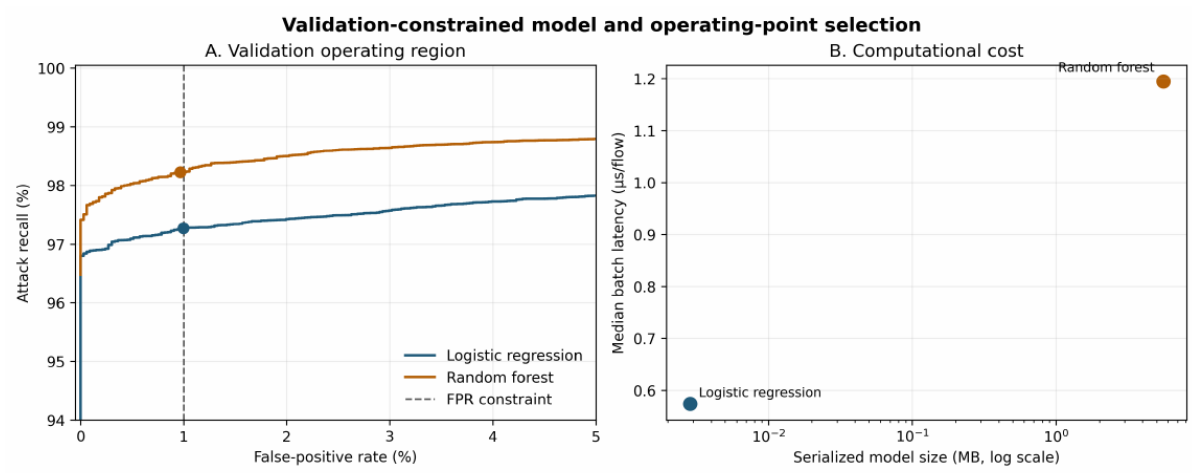
The two class-weighted baselines were Logistic Regression and Random Forest. Within each model, the validation threshold with the highest attack recall under FPR $\leq 1\%$ was retained. If model recalls differed by no more than 0.1 percentage points, the smaller fitted model was preferred. The selected model and threshold were then applied once to the held-out partition.

Default-threshold baselines

Held-out metric	Logistic Regression	Random Forest
Attack recall	96.880%	98.391%
False-positive rate	0.030%	1.843%
MCC	0.705	0.803
Macro average precision	0.885	0.946
Serialized size	2.9 KB	5.56 MB
Median batch latency	0.574 us/flow	1.195 us/flow

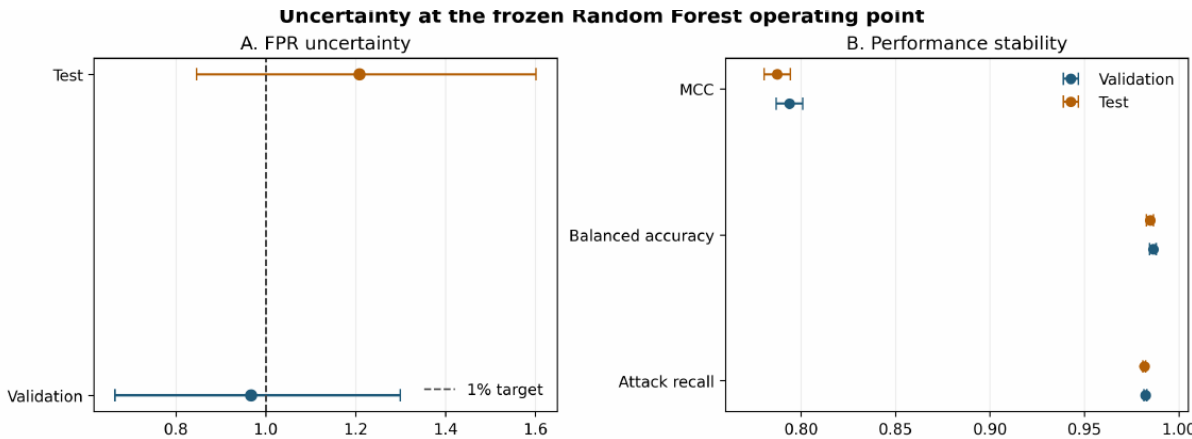
At the default threshold, Random Forest recovered more attacks and achieved higher MCC, while Logistic Regression produced fewer false alarms and occupied substantially less storage. Timing values are retained as measurements from the original run; hardware metadata were not recorded, so the absolute values should not be compared across systems.

Operating-point selection



Random Forest was selected at threshold 0.59853 because its validation recall (98.228%) exceeded the feasible Logistic Regression point (97.266%) by more than the 0.1 percentage-point tie tolerance. On the held-out partition, the new threshold reduced Random Forest false positives from 61 to 40 and increased missed attacks from 1,614 to 1,839 relative to threshold 0.50.

Conditional uncertainty



Held-out metric	Observed	Conditional 95% interval
False-positive rate	1.2085%	0.8459% to 1.6012%
Attack recall	98.1667%	98.0839% to 98.2484%
Balanced accuracy	98.4791%	98.2851% to 98.6620%
MCC	0.7875	0.7806 to 0.7945

Intervals are conditional on the fitted Random Forest, selected threshold, and observed partition. They do not include variation from retraining, repartitioning, or model selection.

Interpretation

The pilot shows a concrete gap between satisfying an operating constraint during validation and retaining the same point estimate on held-out data. It does not show that Random Forest is universally preferable: Logistic Regression offered a much lower false-positive rate and a far smaller model at the default threshold. The appropriate choice depends on the cost assigned to missed attacks, false alarms, memory, and latency.

Limitations

The experiment uses one shard whose records had already been globally shuffled by the dataset producer; it cannot represent a future deployment period. Results come from one fixed random split and one training seed. The binary task also merges heterogeneous attack families into one class. Finally, the timing measurements lack retained hardware metadata and are reported only as run-specific observations.

Next study

In the next study, I would separate devices or time periods, repeat training across seeds, and include an external dataset. I would also extend the decision rule to use an upper confidence bound on false-positive rate together with explicit compute limits. This would test whether adaptive threshold selection remains reliable when traffic composition and available resources change.

Reproducibility

The repository provides the data hash, audit and training scripts, fixed seed, pinned dependencies, preprocessing and split rules, retained result tables, figures, and a notebook with populated outputs. The scripts train and serialize both baselines when the verified dataset shard is placed in the documented location. Raw data are not redistributed.

References

- [1] Neto, E. C. P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., and Ghorbani, A. A. (2023). CICIOT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. *Sensors*, 23(13), 5941. <https://doi.org/10.3390/s23135941>
- [2] Canadian Institute for Cybersecurity, University of New Brunswick. CIC IoT Dataset 2023. <https://www.unb.ca/cic/datasets/iotdataset-2023.html>

Researcher: Wisam Makki Salim | wisam.m.salim@aliraqia.edu.iq